

Petit Rayon de Soleil

Présentation orientée conception, modélisation des données, CI/CD et mise en production.

UML

MERISE

CI/CD

VERCEL + TURSO



Ce que le jury doit pouvoir vérifier

Je montre la chaîne complète : besoin, conception, implémentation, qualité puis mise en production.



Concevoir

besoins, UML, architecture en couches



Modéliser

MCD, MLD, MPD et intégrité des données



Fiabiliser

tests, qualité de code et sécurité



Déployer

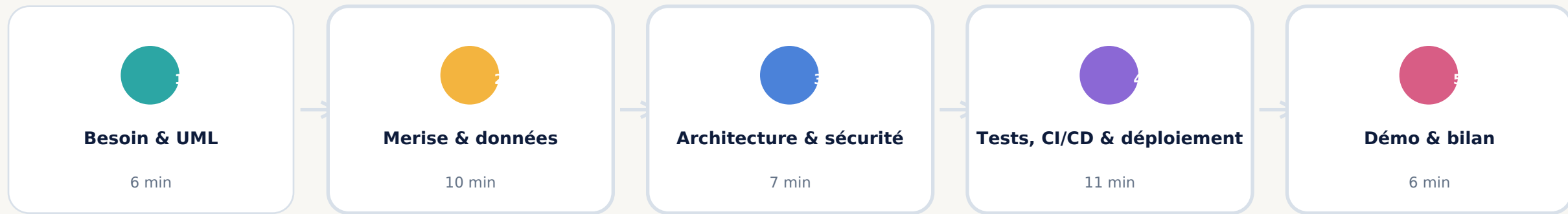
environnements, CI/CD, Vercel et Turso

Point d'attention oral

- Le référentiel attend explicitement le modèle entités-associations, le modèle physique, les cas d'utilisation, le diagramme de séquence, les tests, les scripts et la procédure de déploiement.

40 minutes centrées sur la conception et le déploiement

La démonstration sert de preuve, mais le discours explique d'abord les décisions.



Fil directeur : partir des acteurs et des règles métier, traduire en modèles, puis montrer comment les contrôles sont testés et déployés.

Dans quel ordre faire UML, Merise et la base de données ?

UML et Merise sont complémentaires, ils ne répondent pas à la même question.



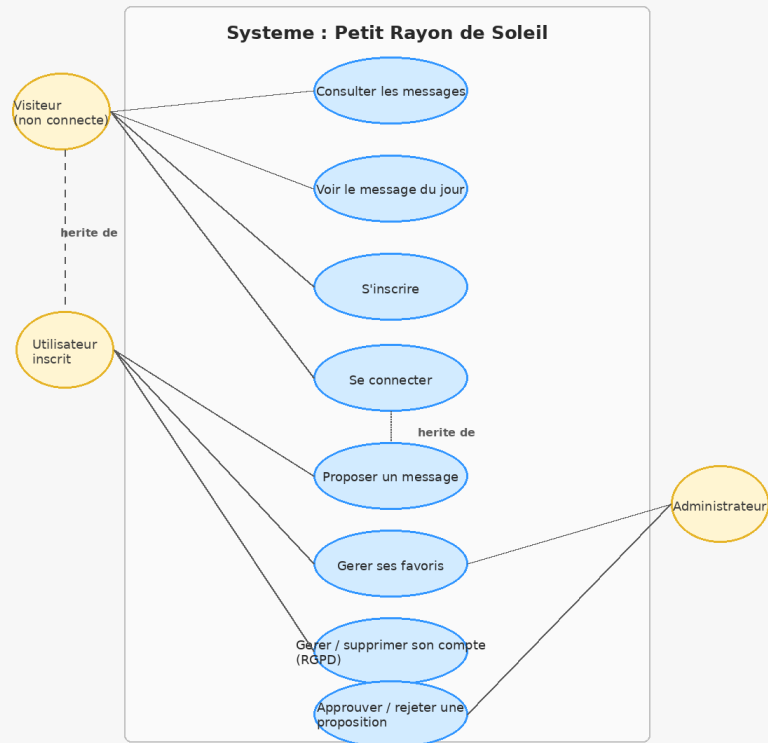
Réponse courte à donner au jury

- Je commence par les besoins et les acteurs. Je modélise les interactions avec UML. Ensuite, je modélise les données avec Merise, du MCD au MPD, avant d'implémenter la base et de tester le scénario le plus important.

Le use case répond à : « qui peut faire quoi ? »

Il fixe le périmètre fonctionnel avant d'entrer dans les détails techniques.

Diagramme de cas d'utilisation



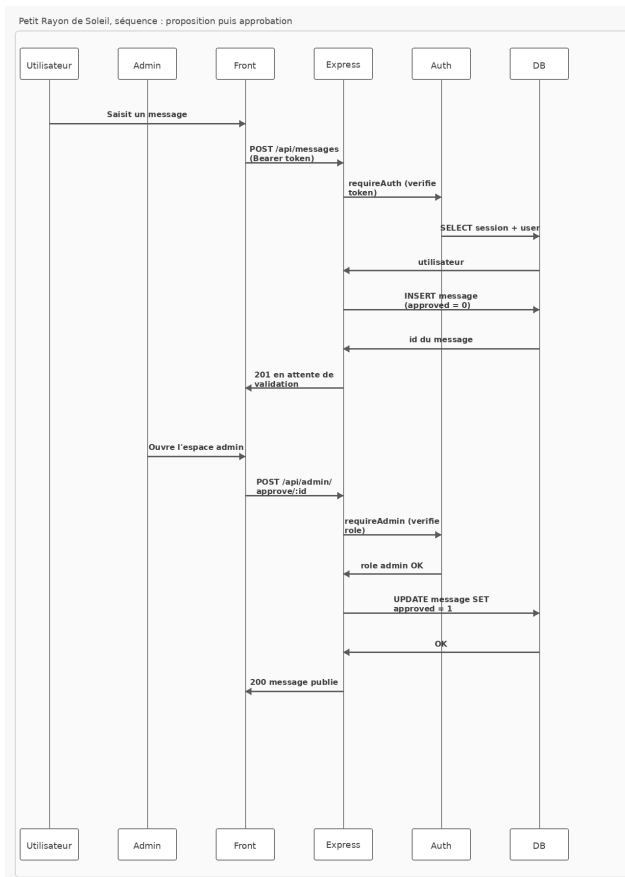
Comment l'expliquer

- Les acteurs externes sont le visiteur, l'utilisateur inscrit et l'administrateur.
- Les ovales représentent les services rendus par le système, non les écrans.
- Le diagramme sert à vérifier la couverture des besoins et les droits par rôle.
- Le cas significatif retenu pour détailler est : proposition puis approbation d'un message.

À ne pas confondre : le use case décrit le besoin fonctionnel, pas les tables, routes ou classes.

Le diagramme de séquence répond à : « dans quel ordre les composants échangent-ils ? »

Il détaille un cas d'utilisation significatif dans le temps.

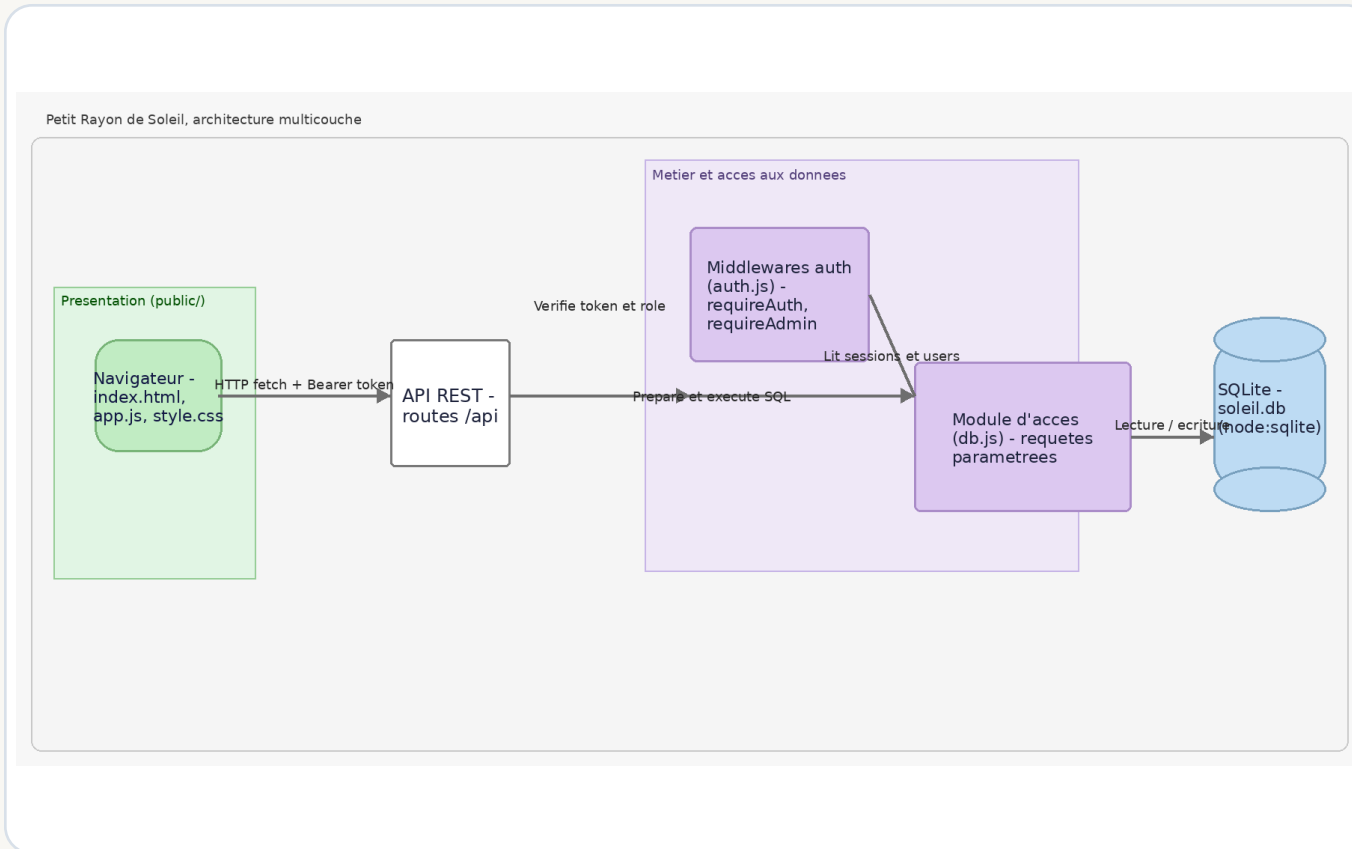


Scénario « proposition puis approbation »

- Le front envoie POST /api/messages avec un Bearer token.
- Express appelle requireAuth puis la base vérifie session et utilisateur.
- Le message est inséré avec approved = 0.
- L'admin appelle ensuite la route d'approbation.
- requireAdmin vérifie le rôle puis la base passe approved à 1.

La conception technique relie les besoins aux responsabilités du code

Chaque couche a un rôle précis et la sécurité reste transversale.



À expliquer au jury

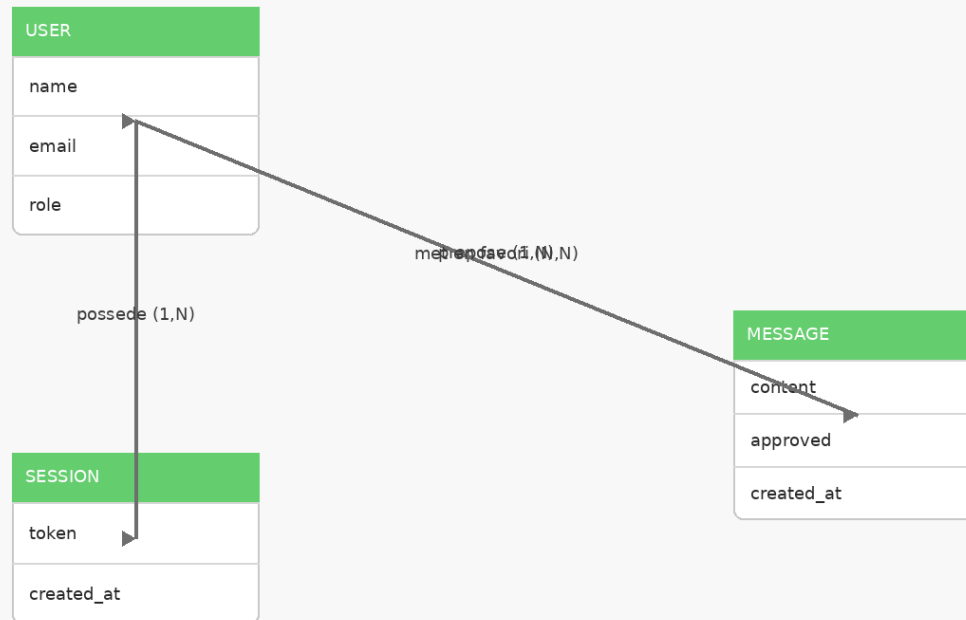
- Présentation : pages, formulaires, navigation et appels HTTP.
- API : routes REST, format des réponses et gestion des erreurs.
- Métier : modération, rôles, validation des règles.
- Données : requêtes paramétrées et contraintes d'intégrité.
- Sécurité : auth et contrôle de rôle au niveau serveur.

Le modèle conceptuel de données, ou MCD

Il décrit les entités métier, les associations et les cardinalités, sans parler de tables ni de SQL.

MCD

MCD — Modele conceptuel de donnees



Lecture du modèle

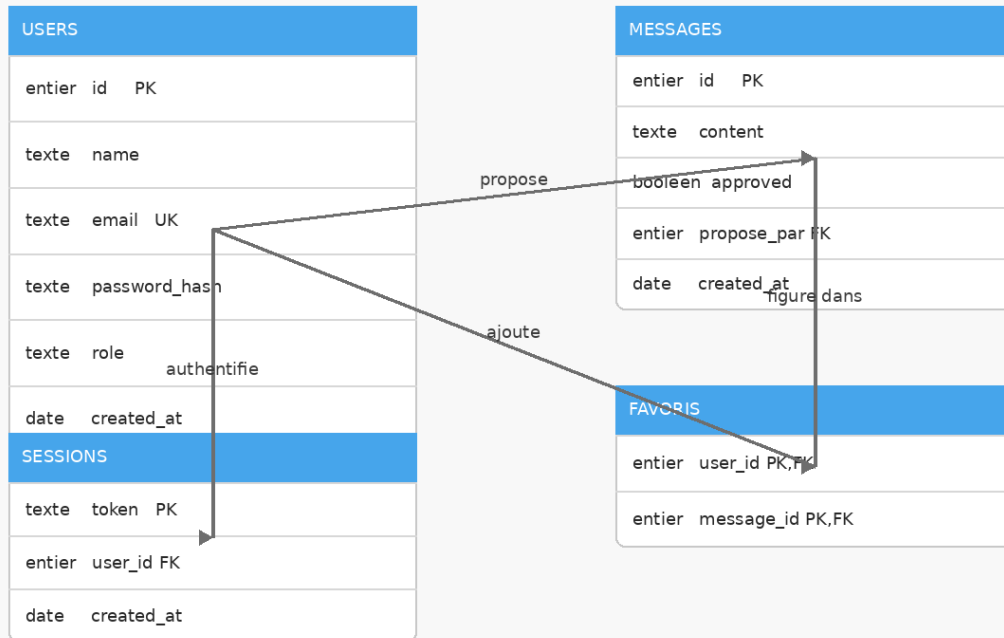
- USER propose des MESSAGE, relation de 1 à N.
- USER possède des SESSION, relation de 1 à N.
- Un USER peut mettre plusieurs MESSAGE en favori et inversement : relation N à N.
- Le MCD traduit le vocabulaire du métier avant tout choix de stockage.

Le modèle logique de données, ou MLD

Il convertit le MCD en structure relationnelle : tables, identifiants, clés primaires et clés étrangères.

MLD

MLD — Modele logique de donnees



Transformation MCD vers MLD

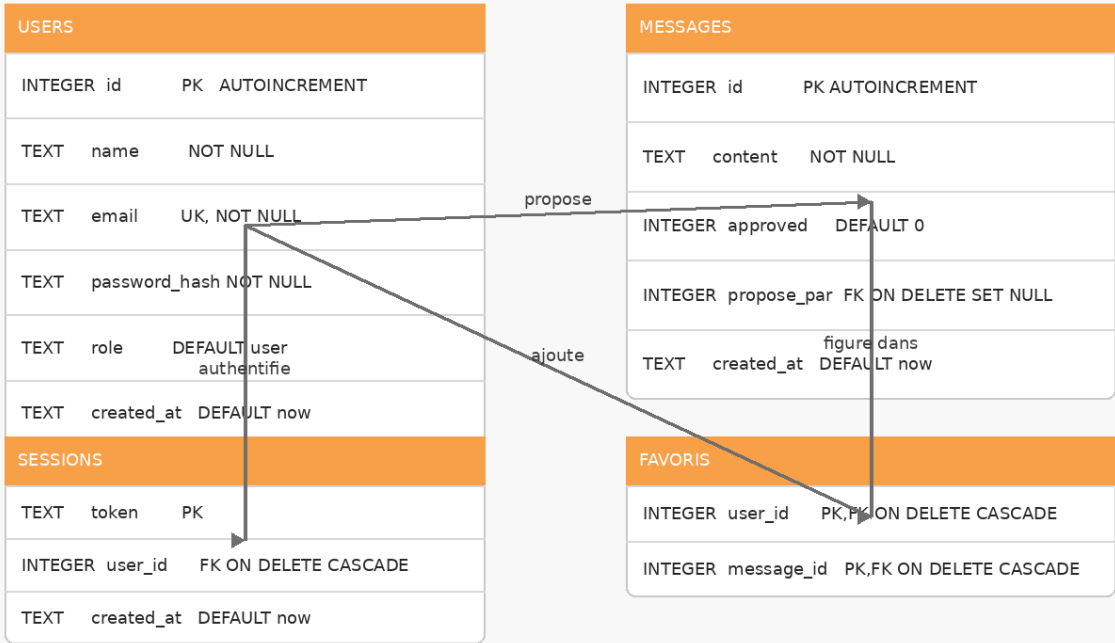
- Les entités deviennent USERS, MESSAGES et SESSIONS.
- La relation 1,N devient une clé étrangère, par exemple propose_par dans MESSAGES.
- La relation N,N « favoris » devient une table d'association FAVORIS.
- Le MLD reste indépendant du moteur de base de données choisi.

Le modèle physique de données, ou MPD

Il est l'implémentation cible pour SQLite : types, contraintes, valeurs par défaut et comportements de suppression.

MPD

MPD — Modele physique de donnees (SQLite)



Points concrets dans SQLite

- USERS.email est unique et obligatoire.
- password_hash est stocké à la place du mot de passe en clair.
- MESSAGES.approved vaut 0 par défaut, donc attente de validation.
- FAVORIS possède une clé primaire composée.
- Les clés étrangères encadrent les suppressions avec CASCADE ou SET NULL.

MCD, MLD et MPD : trois niveaux, une seule continuité

Ce tableau est une réponse directe aux questions techniques probables.

MCD

Niveau : Conceptuel

Question : Métier

Contient : Entités, associations, cardinalités

Ne contient pas : Aucune table, aucune clé SQL

MLD

Niveau : Logique

Question : Relationnel

Contient : Tables, PK, FK, table de liaison

Ne contient pas : Pas de type propre à SQLite

MPD

Niveau : Physique

Question : SGBD

Contient : Types, contraintes, defaults, SQL

Ne contient pas : SQLite, index, CASCADE, UNIQUE

Phrase à retenir : « Le MCD explique le métier, le MLD structure le relationnel, le MPD configure la base réellement exécutée. »

Même code, configurations différentes : développement, test, production

Le but est de tester ce qui sera réellement déployé sans exposer les secrets.



Bascule de configuration

- db.js choisit Turso si les variables de production existent, sinon la base SQLite locale. Le code métier reste le même, seule la configuration change.

Tester le scénario critique et les risques associés

Le parcours proposition puis approbation traverse toutes les couches et les deux rôles.

```
petit-rayon-de-soleil , npm test

$ npm test
> petit-rayon-de-soleil@1.0.0 test
> node --test

Base initialisée avec un admin, un utilisateur et 5 messages.
✓ GET /api/messages/random renvoie un message (144.66ms)
✓ inscription puis connexion fonctionnent (293.67ms)
✓ connexion avec mauvais mot de passe est refusée (119.81ms)
✓ proposer un message sans être connecté est refusé (401) (4.42ms)
✓ un utilisateur normal ne peut pas accéder à l'admin (403) (120.08ms)
✓ parcours complet : proposition -> approbation admin (245.25ms)
✓ suppression de compte (RGPD) efface l'utilisateur (125.88ms)
i tests 7
i pass 7
i fail 0
i duration_ms 1745.69
```

Niveaux et contrôles

- Unitaire : fonctions isolées, par exemple token ou hachage.
- Intégration : route, logique et base reliées via de vraies requêtes HTTP.
- Système et acceptation : parcours complet et vérification manuelle.
- Qualité : ESLint, Prettier et node:test sont rejoués automatiquement dans la CI.

Avant l'oral : afficher le dernier rapport réel et annoncer exactement son total de tests, car des documents antérieurs utilisent des chiffres différents.

CI vérifie, CD livre ou déploie

Le vocabulaire doit être précis : continuous delivery et continuous deployment ne désignent pas forcément la même automatisation.

CI, intégration continue

- Déclenchée à chaque push ou pull request.
- Récupère le code, installe les dépendances, contrôle la qualité et exécute les tests.
- Objectif : détecter rapidement une régression.

Continuous delivery

- Le livrable est prêt à être déployé après validation.
- Une approbation humaine peut rester nécessaire avant la production.
- Objectif : rendre la livraison fiable et répétable.

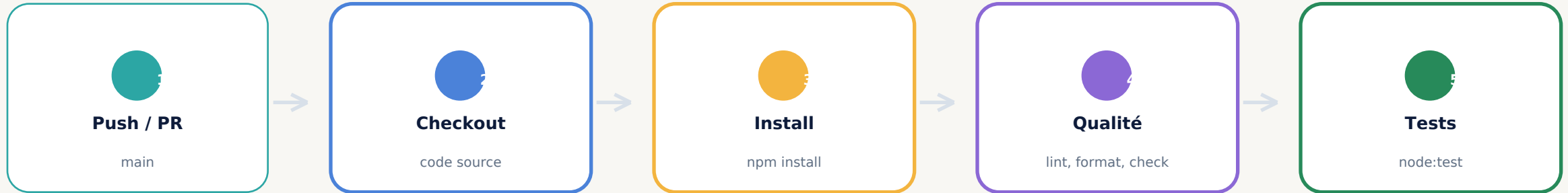
Continuous deployment

- Chaque modification validée est publiée automatiquement en production.
- Dans le projet, Vercel redéploie à partir du dépôt Git.
- Objectif : automatiser aussi la mise en ligne.

Réponse courte : « GitHub Actions est le gardien de la qualité. Vercel est le mécanisme de publication. »

À chaque push ou pull request, le code est vérifié

La CI ne se contente pas de lancer les tests : elle rejoue les contrôles de qualité avant la suite du flux.

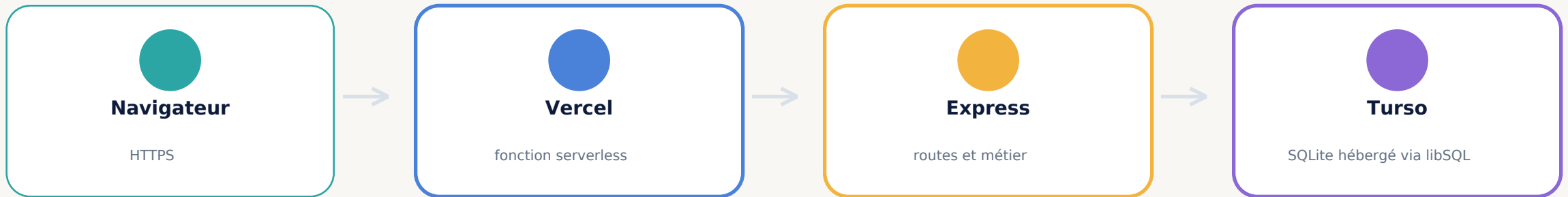


Résultat vert : la qualité et les tests ont réussi.

Résultat rouge : lire le journal, corriger le test ou la règle, puis repousser le code.

Pourquoi Turso est nécessaire en production ?

Vercel est serverless : un fichier SQLite local ne garantit pas une persistance fiable en production.



Sécurité et exploitation

- Les secrets `TURSO_DATABASE_URL` et `TURSO_AUTH_TOKEN` sont définis dans les variables d'environnement, pas dans le code ni dans Git. Vercel est connecté au dépôt GitHub et redéploie après un push sur main.

Ce qui est automatisé, ce qui reste volontairement manuel

L'automatisation sécurise les contrôles répétables, tandis que la décision de changement reste maîtrisée.

Automatisé

- GitHub Actions : installation, qualité et tests à chaque push ou PR.
- Vercel : build et publication depuis la branche principale.
- predeploy : les tests sont relancés avant un déploiement CLI.
- Scripts npm : setup, init-db, test, deploy, redeploy.

Manuel ou à renforcer

- Relecture humaine avant fusion de la PR.
- Migrations et restauration de base.
- Tests visuels de bout en bout dans un navigateur.
- Garde stricte de déploiement uniquement après CI verte, à finaliser si nécessaire.

À expliquer : « Je n'automatise pas aveuglément. J'automatise les répétitions vérifiables et je conserve une décision humaine pour les choix sensibles. »

Démontrer le scénario représentatif, pas seulement l’interface

Le parcours « proposition puis approbation » relie use case, séquence, données, sécurité, test et déploiement.



Connexion utilisateur Montrer la session ouverte et l’accès aux actions membre.



Proposition Soumettre un message, constater l’état en attente.



Contrôle admin Changer de rôle, montrer la route et la validation.



Publication Valider, vérifier approved = 1 et le retour sur l’accueil.



Preuve Montrer le test lié ou le rapport CI vert.

Plan B : garder ouverts les diagrammes, la capture du pipeline et le rap

CONCLUSION

Une démonstration de conception jusqu'à la production.

Les points à maîtriser : UML pour les comportements, Merise pour les données, architecture en couches pour les responsabilités, puis tests, CI/CD et déploiement pour fiabiliser la mise en ligne.

Questions techniques

Références : dossier de projet, annexes UML/Merise, CI/CD, qualité, environnements et déploiement, 2026.